Projeto Integrador: Modelagem de Banco de Dados

LOJA ONLINE DE VENDA DE BLUSAS | DOCUMENTAÇÃO TÉCNICA COMPLETA

Integrantes do Grupo

Nome	RGM
Eduardo de Mattos Borges	04950253-1
Kauã Justino da Silva	50011944
Julia Morales	48208043
Davi Guimarães da Silva	48163872
Mateus Moreira	49827189
Jennifer Santos	04981472-9

Projeto Integrador: Modelagem de Banco de Dados

LOJA ONLINE DE VENDA DE BLUSAS | DOCUMENTAÇÃO TÉCNICA COMPLETA

1. Formulação do Problema e Motivação do Projeto

O comércio de vestuário, especificamente focado na comercialização de blusas, enfrenta desafios operacionais complexos no gerenciamento de inventário e no processamento de pedidos. A principal problemática reside na necessidade de gerenciar o estoque em múltiplos níveis de variação (tamanhos, cores e tecidos) para um mesmo produto base, garantindo que não haja vendas sem estoque (out-of-stock) ou incoerências financeiras no histórico dos pedidos.

Desafios Principais Resolvidos por este Banco de Dados:

- **Controle Fracionado de Estoque (SKU):** Cada variação física da blusa (ex: Tamanho M, Cor Branca) exige uma unidade de manutenção de estoque (SKU) individual.
- **Preservação Histórica de Preços:** Alterações de preço no catálogo de produtos não podem afetar os registros contábeis de pedidos concluídos no passado.
- **Rastreabilidade de Entregas e Pagamentos:** Vínculo direto de cada pedido com o seu respectivo status de pagamento e código de rastreamento logístico.

2. Checklist e Verificação dos Requisitos

Abaixo está a conferência detalhada que atesta a total conformidade deste documento com as especificações e exigências do arquivo do professor (Aula 06 - Projeto Integrador e Aula 09 - SQL com MySQL):

Requisito Solicitado	Aplicação Prática no Projeto da Loja de Blusas	Status
1. Definição do Escopo e Regras de Negócio	Detalhamento completo do funcionamento do e-commerce de blusas: catálogo, variações (tamanho/cor/material), carrinho/pedidos, clientes, pagamento e rastreio.	**Atendido**
2. Modelo Conceitual (DER)	Identificação de todas as entidades, seus atributos identificadores e secundários, além do mapeamento explícito de cardinalidades (1:1, 1:N, N:M).	**Atendido**
3. Modelo Lógico (Esquema Relacional)	Mapeamento para tabelas relacionais com indicação explícita de Chaves Primárias (PK) e Chaves Estrangeiras (FK). Normalizado na 3ª Forma Normal (3FN).	**Atendido**
4. Modelo Físico (Script DDL - MySQL)	Códigos SQL completos de criação das tabelas em MySQL, com tipos de dados adequados, restrições `NOT NULL`, `UNIQUE`, `CHECK`, `FOREIGN KEY` e integridade referencial.	**Atendido**
5. Povoamento Inicial (Script DML)	Inserção de dados fictícios realistas e coerentes para todas as tabelas (10 clientes, 8 produtos, 14 variações, 10 pedidos), permitindo testar a integridade relacional.	**Atendido**
6. Consultas de Teste (Script DQL)	Consultas com junções (`JOIN`), agregações e agrupamentos, além de `UPDATE` / `DELETE` controlados, consultas de auditoria e testes de integridade.	**Atendido**

2. Descrição do Escopo e Regras de Negócio

2.1 Escopo do Sistema

O sistema foi concebido para suportar uma plataforma e-commerce focada na comercialização de blusas femininas e masculinas. O sistema controla a gestão do catálogo de produtos com variações de grade (tamanho, cor, tecido), controle de estoque em tempo real por SKU, cadastro de clientes com múltiplos endereços, fluxo de checkout, processamento de pagamentos e controle de expedição e entrega.

2.2 Regras de Negócio (RN)

- **RN01 - Clientes:** Todo cliente possui um cadastro individual com CPF e E-mail únicos. Um cliente pode cadastrar múltiplos endereços (entrega e cobrança).
- **RN02 - Catálogo e Categorias:** Os produtos (blusas) pertencem obrigatoriamente a uma Categoria (ex: Blusas de Frio, Regatas, Camisas Sociais, Croppeds).
- **RN03 - Variação e SKU:** Um produto blusa possui variações específicas de Tamanho (P, M, G, GG), Cor e Material. Cada variação possui um SKU (Stock Keeping Unit) único e controle individual de quantidade em estoque.
- **RN04 - Pedidos:** Um pedido é associado a um único cliente e a um único endereço de entrega selecionado. Um pedido pode conter um ou vários itens.
- **RN05 - Preservação de Histórico de Preço:** O item do pedido registra a quantidade comprada e o preço unitário praticado no momento da compra, garantindo que alterações futuras no catálogo não alterem o histórico financeiro do pedido.
- **RN06 - Pagamento e Entrega:** Cada pedido aprovado possui um registro de Pagamento (PIX, Cartão ou Boleto) e um registro de Entrega com código de rastreamento e transportadora.

3. Modelo Conceitual (Diagrama Entidade-Relacionamento - DER)

3.1 Entidades e Atributos

- **CLIENTE:** `id_cliente` (PK), nome, cpf, email, telefone, data_nascimento
- **ENDERECO:** `id_endereco` (PK), logradouro, numero, complemento, bairro, cidade, estado, cep, tipo
- **CATEGORIA:** `id_categoria` (PK), nome, descricao
- **PRODUTO:** `id_produto` (PK), nome, descricao, preco_base
- **VARIACAO_PRODUTO:** `id_variacao` (PK), sku, tamanho, cor, material, preco_adicional, quantidade_estoque
- **PEDIDO:** `id_pedido` (PK), data_pedido, status_pedido, valor_total
- **ITEM_PEDIDO:** `id_item_pedido` (PK), quantidade, preco_unitario, subtotal
- **PAGAMENTO:** `id_pagamento` (PK), forma_pagamento, status_pagamento, data_pagamento, valor_pago
- **ENTREGA:** `id_entrega` (PK), codigo_rastreio, transportadora, status_entrega, data_envio, data_entrega_estimada

3.2 Relacionamentos e Cardinalidades

- **CLIENTE (1,N) ---- possui ---- (1,1) ENDERECO:** Um cliente pode ter um ou mais endereços cadastrados; cada endereço pertence obrigatoriamente a um cliente.
- **CLIENTE (1,N) ---- realiza ---- (1,1) PEDIDO:** Um cliente pode realizar vários pedidos; cada pedido pertence a exatamente um cliente.

- **CATEGORIA (1,N) ---- engloba ---- (1,1) PRODUTO:** Uma categoria agrupa vários produtos; cada produto pertence a uma única categoria.
- **PRODUTO (1,N) ---- possui ---- (1,1) VARIACAO_PRODUTO:** Um produto blusa possui uma ou mais variações (tamanhos/cores); cada variação pertence a um produto.
- **PEDIDO (1,N) ---- contem ---- (1,1) ITEM_PEDIDO:** Um pedido contém um ou mais itens; cada item pertence a um único pedido.
- **VARIACAO_PRODUTO (1,N) ---- compoe ---- (1,1) ITEM_PEDIDO:** Uma variação de blusa pode estar em diversos itens de pedidos; cada item refere-se a uma única variação de produto.
- **PEDIDO (1,1) ---- gera ---- (1,1) PAGAMENTO:** Todo pedido concluído possui exatamente um registro de pagamento associado.
- **PEDIDO (1,1) ---- origina ---- (1,1) ENTREGA:** Cada pedido despachado possui exatamente um registro de rastreio e entrega.

4. Modelo Lógico (Esquema Relacional)

O modelo lógico estruturado com suas respectivas Chaves Primárias (PK) e Chaves Estrangeiras (FK):

- **Cliente** (id_cliente, nome, cpf, email, telefone, data_nascimento)
- **Endereco** (id_endereco, logradouro, numero, complemento, bairro, cidade, estado, cep, tipo, id_cliente[FK])
- **Categoria** (id_categoria, nome, descricao)
- **Produto** (id_produto, nome, descricao, preco_base, id_categoria[FK])
- **VariacaoProduto** (id_variacao, sku, tamanho, cor, material, preco_adicional, quantidade_estoque, id_produto[FK])
- **Pedido** (id_pedido, data_pedido, status_pedido, valor_total, id_cliente[FK], id_endereco_entrega[FK])
- **ItemPedido** (id_item_pedido, quantidade, preco_unitario, subtotal, id_pedido[FK], id_variacao[FK])
- **Pagamento** (id_pagamento, forma_pagamento, status_pagamento, data_pagamento, valor_pago, id_pedido[FK])
- **Entrega** (id_entrega, codigo_rastreio, transportadora, status_entrega, data_envio, data_entrega_estimada, id_pedido[FK])

5. Modelo Físico (Script DDL - MySQL)

Script SQL completo para criação das tabelas em MySQL (versão 8 ou superior), com todas as restrições de integridade. O bloco inicial remove as tabelas existentes na ordem correta de dependência, permitindo recriar todo o experimento do zero. Os comandos `CREATE DATABASE` e `USE` ficam comentados porque ambientes online, como o OneCompiler, podem bloqueá-los.

```sql
-- 01. Criacao do banco (quando o ambiente permitir)
-- No OneCompiler estes comandos podem ser bloqueados. Se der erro, ignore.
-- CREATE DATABASE db_loja_blusas;
-- USE db_loja_blusas;

-- Etapa 0 - Limpar o ambiente (ordem: primeiro quem depende dos outros)
DROP TABLE IF EXISTS entrega;
DROP TABLE IF EXISTS pagamento;
DROP TABLE IF EXISTS item_pedido;
DROP TABLE IF EXISTS pedido;
DROP TABLE IF EXISTS variacao_produto;
DROP TABLE IF EXISTS produto;
DROP TABLE IF EXISTS categoria;
DROP TABLE IF EXISTS endereco;
DROP TABLE IF EXISTS cliente;
```

```sql
-- Etapa 1 - Criar CLIENTE
CREATE TABLE cliente (
    id_cliente INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    cpf VARCHAR(14) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    telefone VARCHAR(20),
    data_nascimento DATE
);

-- Etapa 2 - Criar ENDERECO (um cliente pode ter varios enderecos)
CREATE TABLE endereco (
    id_endereco INT PRIMARY KEY AUTO_INCREMENT,
    id_cliente INT NOT NULL,
    logradouro VARCHAR(150) NOT NULL,
    numero VARCHAR(20) NOT NULL,
    complemento VARCHAR(50),
    bairro VARCHAR(50) NOT NULL,
    cidade VARCHAR(50) NOT NULL,
    estado CHAR(2) NOT NULL,
    cep VARCHAR(10) NOT NULL,
    tipo VARCHAR(20) DEFAULT 'Entrega',
    CONSTRAINT fk_endereco_cliente
        FOREIGN KEY (id_cliente)
        REFERENCES cliente(id_cliente)
        ON DELETE CASCADE
);

-- Etapa 3 - Criar CATEGORIA
CREATE TABLE categoria (
    id_categoria INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(50) NOT NULL,
    descricao TEXT
);

-- Etapa 4 - Criar PRODUTO (blusa "base", sem tamanho/cor)
CREATE TABLE produto (
    id_produto INT PRIMARY KEY AUTO_INCREMENT,
    id_categoria INT NOT NULL,
    nome VARCHAR(100) NOT NULL,
    descricao TEXT,
    preco_base DECIMAL(10,2) NOT NULL,
    CHECK (preco_base >= 0),
    CONSTRAINT fk_produto_categoria
        FOREIGN KEY (id_categoria)
        REFERENCES categoria(id_categoria)
);

-- Etapa 5 - Criar VARIACAO_PRODUTO (cada linha = um SKU com estoque proprio)
CREATE TABLE variacao_produto (
    id_variacao INT PRIMARY KEY AUTO_INCREMENT,
    id_produto INT NOT NULL,
    sku VARCHAR(50) NOT NULL UNIQUE,
    tamanho VARCHAR(10) NOT NULL,
    cor VARCHAR(30) NOT NULL,
    material VARCHAR(50),
    preco_adicional DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    quantidade_estoque INT NOT NULL DEFAULT 0,
    CHECK (preco_adicional >= 0),
    CHECK (quantidade_estoque >= 0),
    CONSTRAINT fk_variacao_produto
        FOREIGN KEY (id_produto)
        REFERENCES produto(id_produto)
        ON DELETE CASCADE
);

-- Etapa 6 - Criar PEDIDO
CREATE TABLE pedido (
    id_pedido INT PRIMARY KEY AUTO_INCREMENT,
    id_cliente INT NOT NULL,
    id_endereco_entrega INT NOT NULL,
    data_pedido TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status_pedido VARCHAR(30) NOT NULL DEFAULT 'Pendente',
    valor_total DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    CHECK (valor_total >= 0),
    CONSTRAINT fk_pedido_cliente
```

```
        FOREIGN KEY (id_cliente)
        REFERENCES cliente(id_cliente),
    CONSTRAINT fk_pedido_endereco
        FOREIGN KEY (id_endereco_entrega)
        REFERENCES endereco(id_endereco)
);

-- Etapa 7 - Criar ITEM_PEDIDO (guarda o preco praticado na compra)
CREATE TABLE item_pedido (
    id_item_pedido INT PRIMARY KEY AUTO_INCREMENT,
    id_pedido INT NOT NULL,
    id_variacao INT NOT NULL,
    quantidade INT NOT NULL,
    preco_unitario DECIMAL(10,2) NOT NULL,
    subtotal DECIMAL(10,2) NOT NULL,
    CHECK (quantidade > 0),
    CHECK (preco_unitario >= 0),
    CONSTRAINT fk_item_pedido
        FOREIGN KEY (id_pedido)
        REFERENCES pedido(id_pedido)
        ON DELETE CASCADE,
    CONSTRAINT fk_item_variacao
        FOREIGN KEY (id_variacao)
        REFERENCES variacao_produto(id_variacao)
);

-- Etapa 8 - Criar PAGAMENTO (1 pedido = 1 pagamento)
CREATE TABLE pagamento (
    id_pagamento INT PRIMARY KEY AUTO_INCREMENT,
    id_pedido INT NOT NULL UNIQUE,
    forma_pagamento VARCHAR(30) NOT NULL,
    status_pagamento VARCHAR(30) NOT NULL,
    data_pagamento TIMESTAMP NULL,
    valor_pago DECIMAL(10,2) NOT NULL,
    CHECK (valor_pago >= 0),
    CONSTRAINT fk_pagamento_pedido
        FOREIGN KEY (id_pedido)
        REFERENCES pedido(id_pedido)
);

-- Etapa 9 - Criar ENTREGA (1 pedido = 1 entrega)
CREATE TABLE entrega (
    id_entrega INT PRIMARY KEY AUTO_INCREMENT,
    id_pedido INT NOT NULL UNIQUE,
    codigo_rastreio VARCHAR(50),
    transportadora VARCHAR(50),
    status_entrega VARCHAR(30) DEFAULT 'Em Processamento',
    data_envio TIMESTAMP NULL,
    data_entrega_estimada DATE,
    CONSTRAINT fk_entrega_pedido
        FOREIGN KEY (id_pedido)
        REFERENCES pedido(id_pedido)
);

-- Para verificar a estrutura:
SHOW TABLES;
DESC variacao_produto;
```

6. Povoamento Inicial de Dados (Script DML)

Inserção de massa de dados fictícia para validação das restrições e relacionamentos do sistema. Os dados cobrem as situações de teste: cliente com mais de um endereço e vários pedidos, blusas com variações de tamanho e cor, pedidos com status diferentes (pago, enviado, aguardando pagamento e cancelado) e as três formas de pagamento.

```sql
-- MASSA DE DADOS (Fase 6)
-- Etapa 10 - Clientes (10)
INSERT INTO cliente (nome, cpf, email, telefone, data_nascimento) VALUES
('Ana Silva',       '000.000.000-01', 'ana.silva@exemplo.com',      '(11) 98765-4321', '1995-04-12'),
('Carlos Eduardo',  '000.000.000-02', 'carlos.edu@exemplo.com',     '(21) 99887-6655', '1988-11-23'),
('Marina Alves',    '000.000.000-03', 'marina.alves@exemplo.com',   '(11) 91111-1003', '1992-02-08'),
('Pedro Santos',    '000.000.000-04', 'pedro.santos@exemplo.com',   '(11) 91111-1004', '1990-07-30'),
('Juliana Rocha',   '000.000.000-05', 'juliana.rocha@exemplo.com',  '(31) 91111-1005', '1998-12-01'),
('Rafael Costa',    '000.000.000-06', 'rafael.costa@exemplo.com',   '(41) 91111-1006', '1985-05-17'),
('Beatriz Nunes',   '000.000.000-07', 'beatriz.nunes@exemplo.com',  '(85) 91111-1007', '2000-09-25'),
('Lucas Martins',   '000.000.000-08', 'lucas.martins@exemplo.com',  '(71) 91111-1008', '1993-03-14'),
('Camila Ferreira', '000.000.000-09', 'camila.ferreira@exemplo.com','(51) 91111-1009', '1997-08-19'),
('Joao Ribeiro',    '000.000.000-10', 'joao.ribeiro@exemplo.com',   '(62) 91111-1010', '1991-01-05');

-- Etapa 11 - Enderecos (11: a Ana tem dois)
INSERT INTO endereco (id_cliente, logradouro, numero, complemento, bairro, cidade, estado, cep, tipo) VALUES
(1,  'Rua das Flores',    '123',  'Apt 42',  'Jardins',    'Sao Paulo',      'SP', '01234-567', 'Entrega'),
(2,  'Av. Atlantica',     '500',  NULL,      'Copacabana', 'Rio de Janeiro', 'RJ', '22070-000', 'Entrega'),
(3,  'Rua Augusta',       '1500', 'Sala 3',  'Consolacao', 'Sao Paulo',      'SP', '01304-001', 'Entrega'),
(4,  'Av. Paulista',      '900',  NULL,      'Bela Vista', 'Sao Paulo',      'SP', '01310-100', 'Entrega'),
(5,  'Rua da Bahia',      '250',  'Apt 12',  'Lourdes',    'Belo Horizonte', 'MG', '30160-011', 'Entrega'),
(6,  'Rua XV de Novembro','80',   NULL,      'Centro',     'Curitiba',       'PR', '80020-310', 'Entrega'),
(7,  'Av. Beira Mar',     '1200', 'Apt 801', 'Meireles',   'Fortaleza',      'CE', '60165-121', 'Entrega'),
(8,  'Rua Chile',         '45',   NULL,      'Centro',     'Salvador',       'BA', '40020-000', 'Entrega'),
(9,  'Rua dos Andradas',  '300',  NULL,      'Centro',     'Porto Alegre',   'RS', '90020-007', 'Entrega'),
(10, 'Av. Brasil',        '700',  'Casa 2',  'Centro',     'Goiania',        'GO', '74003-010', 'Entrega'),
(1,  'Rua Oscar Freire',  '700',  'Sala 10', 'Pinheiros',  'Sao Paulo',      'SP', '05409-010', 'Cobranca');

-- Etapa 12 - Categorias (4)
INSERT INTO categoria (nome, descricao) VALUES
('Blusas de Frio',  'Moletons, tricots, cardigans e casacos leves'),
('Regatas',         'Blusas sem manga para o dia a dia e dias quentes'),
('Camisas Sociais', 'Blusas sociais com botao e gola para trabalho'),
('Croppeds',        'Blusas curtas para o dia a dia');

-- Etapa 13 - Produtos (8)
INSERT INTO produto (id_categoria, nome, descricao, preco_base) VALUES
(1, 'Blusa Tricot Gola Alta',   'Blusa de tricot macia e aquecida',          120.00),
(1, 'Moletom Canguru',          'Moletom com capuz e bolso frontal',         159.90),
(2, 'Regata de Cetim Basic',    'Regata leve com toque acetinado',            59.90),
(2, 'Regata Canelada',          'Regata de malha canelada ajustada',          39.90),
(3, 'Camisa Social Linho',      'Camisa de linho manga longa botao frontal', 149.90),
(3, 'Camisa Social Slim',       'Camisa de algodao com corte slim',          129.90),
(4, 'Cropped Tricot',           'Cropped de tricot com gola redonda',         69.90),
(4, 'Cropped Malha Listrado',   'Cropped de malha com listras',               49.90);

-- Etapa 14 - Variacoes / SKU (14)
INSERT INTO variacao_produto (id_produto, sku, tamanho, cor, material, preco_adicional, quantidade_estoque) VALUES
(1, 'TRI-GA-BR-P',  'P', 'Branca', 'Algodao/La',  0.00, 15),
(1, 'TRI-GA-BR-M',  'M', 'Branca', 'Algodao/La',  0.00, 20),
(1, 'TRI-GA-PR-G',  'G', 'Preta',  'Algodao/La', 10.00, 10),
(2, 'MOL-CAN-CZ-M', 'M', 'Cinza',  'Algodao',     0.00, 25),
(2, 'MOL-CAN-PR-G', 'G', 'Preta',  'Algodao',     0.00, 18),
(3, 'REG-CET-AZ-M', 'M', 'Azul',   'Seda/Cetim',  0.00, 30),
(3, 'REG-CET-RS-P', 'P', 'Rosa',   'Seda/Cetim',  0.00, 22),
(4, 'REG-CAN-BR-M', 'M', 'Branca', 'Algodao',     0.00, 40),
(5, 'CAM-LIN-BC-G', 'G', 'Bege',   'Linho',       0.00,  8),
(5, 'CAM-LIN-BC-M', 'M', 'Bege',   'Linho',       0.00, 12),
(6, 'CAM-SLI-AZ-G', 'G', 'Azul',   'Algodao',     0.00, 14),
(7, 'CRO-TRI-RS-P', 'P', 'Rosa',   'Tricot',      0.00, 16),
(7, 'CRO-TRI-PR-M', 'M', 'Preta',  'Tricot',      5.00,  9),
(8, 'CRO-LIS-BR-P', 'P', 'Branca', 'Malha',       0.00, 20);

-- Etapa 15 - Pedidos (10)
-- Atencao: o valor_total precisa bater com a soma dos itens.
```

```sql
INSERT INTO pedido (id_cliente, id_endereco_entrega, data_pedido, status_pedido, valor_total) VALUES
(1, 1, '2026-10-01 10:15:00', 'Pago',                179.90),
(2, 2, '2026-10-03 14:30:00', 'Pago',                359.70),
(1, 1, '2026-10-08 09:05:00', 'Pago',                279.90),
(3, 3, '2026-10-12 16:45:00', 'Pago',                189.70),
(4, 4, '2026-10-15 11:20:00', 'Enviado',             279.80),
(5, 5, '2026-10-20 18:10:00', 'Pago',                149.80),
(6, 6, '2026-10-25 20:00:00', 'Aguardando Pagamento', 360.00),
(7, 7, '2026-10-28 13:40:00', 'Pago',                279.40),
(1, 1, '2026-11-02 08:50:00', 'Pago',                219.80),
(8, 8, '2026-11-05 15:25:00', 'Cancelado',           149.90);

-- Etapa 16 - Itens dos pedidos
INSERT INTO item_pedido (id_pedido, id_variacao, quantidade, preco_unitario, subtotal) VALUES
(1, 1,  1, 120.00, 120.00),   -- Tricot P Branca
(1, 6,  1,  59.90,  59.90),   -- Regata Cetim M Azul
(2, 4,  2, 159.90, 319.80),   -- Moletom M Cinza
(2, 8,  1,  39.90,  39.90),   -- Regata Canelada M Branca
(3, 3,  1, 130.00, 130.00),   -- Tricot G Preta (120 + 10)
(3, 9,  1, 149.90, 149.90),   -- Camisa Linho G Bege
(4, 7,  2,  59.90, 119.80),   -- Regata Cetim P Rosa
(4, 12, 1,  69.90,  69.90),   -- Cropped Tricot P Rosa
(5, 10, 1, 149.90, 149.90),   -- Camisa Linho M Bege
(5, 11, 1, 129.90, 129.90),   -- Camisa Slim G Azul
(6, 13, 2,  74.90, 149.80),   -- Cropped Tricot M Preta (69,90 + 5)
(7, 2,  3, 120.00, 360.00),   -- Tricot M Branca
(8, 14, 4,  49.90, 199.60),   -- Cropped Listrado P Branca
(8, 8,  2,  39.90,  79.80),   -- Regata Canelada M Branca
(9, 5,  1, 159.90, 159.90),   -- Moletom G Preta
(9, 7,  1,  59.90,  59.90),   -- Regata Cetim P Rosa
(10, 9, 1, 149.90, 149.90);   -- Camisa Linho G Bege

-- Etapa 17 - Pagamentos (1 por pedido)
INSERT INTO pagamento (id_pedido, forma_pagamento, status_pagamento, data_pagamento, valor_pago) VALUES
(1,  'PIX',    'Aprovado',  '2026-10-01 10:16:00', 179.90),
(2,  'Cartao', 'Aprovado',  '2026-10-03 14:31:00', 359.70),
(3,  'PIX',    'Aprovado',  '2026-10-08 09:06:00', 279.90),
(4,  'Cartao', 'Aprovado',  '2026-10-12 16:46:00', 189.70),
(5,  'Boleto', 'Aprovado',  '2026-10-16 10:00:00', 279.80),
(6,  'PIX',    'Aprovado',  '2026-10-20 18:11:00', 149.80),
(7,  'Boleto', 'Pendente',  NULL,                    0.00),
(8,  'Cartao', 'Aprovado',  '2026-10-28 13:41:00', 279.40),
(9,  'PIX',    'Aprovado',  '2026-11-02 08:51:00', 219.80),
(10, 'PIX',    'Cancelado', NULL,                    0.00);

-- Etapa 18 - Entregas (pedidos pendentes/cancelados nao tem entrega)
INSERT INTO entrega (id_pedido, codigo_rastreio, transportadora, status_entrega, data_envio, data_entrega_estimada)
VALUES
(1, 'BR123456789BR', 'Correios',  'Entregue',          '2026-10-02 09:00:00', '2026-10-08'),
(2, 'BR987654321BR', 'Correios',  'Entregue',          '2026-10-04 10:00:00', '2026-10-10'),
(3, 'LG555000111XX', 'Loggi',     'Entregue',          '2026-10-09 08:30:00', '2026-10-13'),
(4, 'BR222333444BR', 'Correios',  'Em Transito',       '2026-10-13 09:30:00', '2026-11-10'),
(5, 'JT777888999YY', 'Jadlog',    'Em Transito',       '2026-10-17 11:00:00', '2026-11-12'),
(6, NULL,            NULL,        'Em Processamento',  NULL,                  '2026-11-15'),
(8, 'LG111222333XX', 'Loggi',     'Entregue',          '2026-10-29 10:00:00', '2026-11-04'),
(9, NULL,            NULL,        'Em Processamento',  NULL,                  '2026-11-12');
```

7. Consultas de Validação e Relatórios (Script DQL)

Consultas de validação organizadas em etapas: consultas básicas, junções, alterações controladas (`UPDATE` e `DELETE` com `WHERE`), consultas gerenciais e de auditoria.

```sql
-- CONSULTAS BASICAS
-- Etapa 19 - Primeiro SELECT
SELECT * FROM cliente;

-- Etapa 20 - SELECT com campos especificos
SELECT nome, preco_base
FROM produto;

-- Etapa 21 - Filtrar com WHERE
SELECT nome, preco_base
FROM produto
```

```sql
WHERE preco_base > 100;

-- Etapa 22 - WHERE com AND
SELECT sku, tamanho, cor, quantidade_estoque
FROM variacao_produto
WHERE cor = 'Branca'
  AND quantidade_estoque > 15;

-- Etapa 23 - Pesquisa textual com LIKE
SELECT *
FROM cliente
WHERE nome LIKE 'A%';

-- Etapa 24 - Ordenar com ORDER BY
SELECT nome, preco_base
FROM produto
ORDER BY preco_base DESC;

-- JOINs
-- Etapa 25 - JOIN sem alias (pedido + cliente)
SELECT
    pedido.id_pedido,
    pedido.data_pedido,
    cliente.nome
FROM pedido
INNER JOIN cliente
    ON pedido.id_cliente = cliente.id_cliente
ORDER BY pedido.id_pedido;

-- Etapa 26 - O mesmo JOIN com alias
SELECT
    p.id_pedido,
    p.data_pedido,
    c.nome
FROM pedido p
INNER JOIN cliente c
    ON p.id_cliente = c.id_cliente
ORDER BY p.id_pedido;

-- Etapa 27 - Catalogo completo: produto + categoria + variacoes + estoque
SELECT
    pr.nome AS blusa,
    ca.nome AS categoria,
    v.sku,
    v.tamanho,
    v.cor,
    (pr.preco_base + v.preco_adicional) AS preco_final,
    v.quantidade_estoque
FROM produto pr
INNER JOIN categoria ca
    ON pr.id_categoria = ca.id_categoria
INNER JOIN variacao_produto v
    ON pr.id_produto = v.id_produto
ORDER BY ca.nome, pr.nome, v.tamanho;

-- Etapa 28 - Relatorio detalhado de compras (todas as tabelas)
SELECT
    p.id_pedido,
    p.data_pedido,
    p.status_pedido,
    c.nome AS cliente,
    pr.nome AS blusa,
    v.tamanho,
    v.cor,
    ip.quantidade,
    ip.preco_unitario,
    ip.quantidade * ip.preco_unitario AS subtotal,
    pg.forma_pagamento,
    pg.status_pagamento,
    e.codigo_rastreio,
    e.status_entrega
FROM item_pedido ip
INNER JOIN pedido p
    ON ip.id_pedido = p.id_pedido
INNER JOIN cliente c
    ON p.id_cliente = c.id_cliente
INNER JOIN variacao_produto v
```

```sql
    ON ip.id_variacao = v.id_variacao
INNER JOIN produto pr
    ON v.id_produto = pr.id_produto
LEFT JOIN pagamento pg
    ON p.id_pedido = pg.id_pedido
LEFT JOIN entrega e
    ON p.id_pedido = e.id_pedido
ORDER BY p.id_pedido, pr.nome;

-- Etapa 29 - Compras de um cliente especifico (cliente 1)
SELECT
    c.nome AS cliente,
    p.id_pedido,
    p.data_pedido,
    pr.nome AS blusa,
    v.tamanho,
    v.cor,
    ip.quantidade,
    ip.preco_unitario,
    pg.forma_pagamento,
    e.codigo_rastreio
FROM cliente c
INNER JOIN pedido p
    ON c.id_cliente = p.id_cliente
INNER JOIN item_pedido ip
    ON p.id_pedido = ip.id_pedido
INNER JOIN variacao_produto v
    ON ip.id_variacao = v.id_variacao
INNER JOIN produto pr
    ON v.id_produto = pr.id_produto
LEFT JOIN pagamento pg
    ON p.id_pedido = pg.id_pedido
LEFT JOIN entrega e
    ON p.id_pedido = e.id_pedido
WHERE c.id_cliente = 1;

-- UPDATE e DELETE com seguranca
-- Etapa 30 - Consultar antes de alterar
SELECT * FROM produto WHERE id_produto = 1;

-- Etapa 31 - UPDATE (Tricot Gola Alta passa de 120,00 para 135,00)
UPDATE produto
SET preco_base = 135.00
WHERE id_produto = 1;

SELECT * FROM produto WHERE id_produto = 1;

-- Etapa 32 - Verificar preco historico (RN05)
-- preco_base mudou, mas preco_unitario dos pedidos antigos continua igual.
SELECT
    pr.nome,
    pr.preco_base AS preco_base_atual,
    ip.id_pedido,
    ip.preco_unitario AS preco_pago_na_epoca
FROM produto pr
INNER JOIN variacao_produto v
    ON pr.id_produto = v.id_produto
INNER JOIN item_pedido ip
    ON v.id_variacao = ip.id_variacao
WHERE pr.id_produto = 1
ORDER BY ip.id_pedido;

-- Etapa 33 - UPDATE de estoque (baixa de 1 unidade da variacao 9)
UPDATE variacao_produto
SET quantidade_estoque = quantidade_estoque - 1
WHERE id_variacao = 9;

SELECT sku, quantidade_estoque FROM variacao_produto WHERE id_variacao = 9;

-- Etapa 34 - DELETE controlado (cliente criado so para teste, sem pedidos)
INSERT INTO cliente (nome, cpf, email, telefone)
VALUES ('Cliente Temporario', '000.000.000-99', 'temporario@exemplo.com', '(11) 90000-0000');

SELECT * FROM cliente WHERE email = 'temporario@exemplo.com';

DELETE FROM cliente
WHERE email = 'temporario@exemplo.com';
```

```sql
SELECT * FROM cliente WHERE email = 'temporario@exemplo.com';

-- CONSULTAS GERENCIAIS
-- Etapa 35 - Quantos pedidos existem?
SELECT COUNT(*) AS quantidade_pedidos
FROM pedido;

-- Etapa 36 - Faturamento total (somente pedidos com pagamento aprovado)
SELECT SUM(valor_pago) AS faturamento_aprovado
FROM pagamento
WHERE status_pagamento = 'Aprovado';

-- Etapa 37 - Preco medio do catalogo
SELECT AVG(preco_base) AS preco_medio
FROM produto;

-- Etapa 38 - Quanto cada blusa vendeu? (GROUP BY)
SELECT
    pr.id_produto,
    pr.nome,
    SUM(ip.quantidade) AS unidades_vendidas,
    SUM(ip.quantidade * ip.preco_unitario) AS faturamento
FROM item_pedido ip
INNER JOIN variacao_produto v
    ON ip.id_variacao = v.id_variacao
INNER JOIN produto pr
    ON v.id_produto = pr.id_produto
GROUP BY pr.id_produto, pr.nome
ORDER BY faturamento DESC;

-- Etapa 39 - Faturamento por categoria
SELECT
    ca.nome AS categoria,
    SUM(ip.quantidade) AS unidades_vendidas,
    SUM(ip.quantidade * ip.preco_unitario) AS faturamento
FROM item_pedido ip
INNER JOIN variacao_produto v
    ON ip.id_variacao = v.id_variacao
INNER JOIN produto pr
    ON v.id_produto = pr.id_produto
INNER JOIN categoria ca
    ON pr.id_categoria = ca.id_categoria
GROUP BY ca.id_categoria, ca.nome
ORDER BY faturamento DESC;

-- Etapa 40 - Quanto cada cliente comprou? (ranking)
SELECT
    c.id_cliente,
    c.nome,
    COUNT(DISTINCT p.id_pedido) AS quantidade_pedidos,
    SUM(ip.quantidade * ip.preco_unitario) AS total_comprado
FROM cliente c
INNER JOIN pedido p
    ON c.id_cliente = p.id_cliente
INNER JOIN item_pedido ip
    ON p.id_pedido = ip.id_pedido
GROUP BY c.id_cliente, c.nome
ORDER BY total_comprado DESC;

-- Etapa 41 - Ticket medio
SELECT AVG(total_pedido) AS ticket_medio
FROM (
    SELECT
        id_pedido,
        SUM(quantidade * preco_unitario) AS total_pedido
    FROM item_pedido
    GROUP BY id_pedido
) totais;

-- Etapa 42 - Formas de pagamento mais usadas
SELECT
    forma_pagamento,
    COUNT(*) AS quantidade
FROM pagamento
GROUP BY forma_pagamento
ORDER BY quantidade DESC;
```

```sql
-- CONSULTAS DE AUDITORIA
-- Etapa 43 - Variacoes com estoque baixo (menos de 10 unidades)
SELECT sku, tamanho, cor, quantidade_estoque
FROM variacao_produto
WHERE quantidade_estoque < 10
ORDER BY quantidade_estoque;

-- Etapa 44 - Pedidos cujo valor_total NAO bate com a soma dos itens
-- (se a massa estiver coerente, o resultado deve vir vazio)
SELECT
    p.id_pedido,
    p.valor_total,
    SUM(ip.subtotal) AS soma_itens
FROM pedido p
INNER JOIN item_pedido ip
    ON p.id_pedido = ip.id_pedido
GROUP BY p.id_pedido, p.valor_total
HAVING p.valor_total <> SUM(ip.subtotal);

-- Etapa 45 - Pedidos sem entrega registrada
SELECT p.id_pedido, p.status_pedido, c.nome
FROM pedido p
INNER JOIN cliente c
    ON p.id_cliente = c.id_cliente
LEFT JOIN entrega e
    ON p.id_pedido = e.id_pedido
WHERE e.id_entrega IS NULL;
```

7.1 Testes de Integridade

Os testes abaixo provocam erros de propósito para comprovar que as restrições protegem o banco. Execute um de cada vez; todos devem ser rejeitados.

```sql
-- TESTES DE INTEGRIDADE (provocar erros de proposito)
-- Execute UM de cada vez: cada um deve ser REJEITADO pelo banco.
-- Teste 1 - CPF/e-mail duplicado (UNIQUE)
INSERT INTO cliente (nome, cpf, email, telefone)
VALUES ('Outra Ana', '000.000.000-55', 'ana.silva@exemplo.com', '(11) 99999-9999');

-- Teste 2 - Nome ausente (NOT NULL)
INSERT INTO cliente (cpf, email, telefone)
VALUES ('000.000.000-56', 'semnome@exemplo.com', '(11) 98888-8888');

-- Teste 3 - Quantidade invalida (CHECK)
INSERT INTO item_pedido (id_pedido, id_variacao, quantidade, preco_unitario, subtotal)
VALUES (1, 3, 0, 130.00, 0.00);

-- Teste 4 - Cliente inexistente (FOREIGN KEY)
INSERT INTO pedido (id_cliente, id_endereco_entrega, status_pedido, valor_total)
VALUES (999, 1, 'Pendente', 100.00);

-- Teste 5 - SKU duplicado (UNIQUE)
INSERT INTO variacao_produto (id_produto, sku, tamanho, cor, material, preco_adicional, quantidade_estoque)
VALUES (1, 'TRI-GA-BR-P', 'P', 'Branca', 'Algodao/La', 0.00, 5);
```